

Self-Supervised Flow Matching for Scalable Multi-Modal Synthesis

Hila Chefer, Patrick Esser, Dominik Lorenz, Dustin Podell,
Vikash Raja, Vinh Tong, Antonio Torralba, Robin Rombach

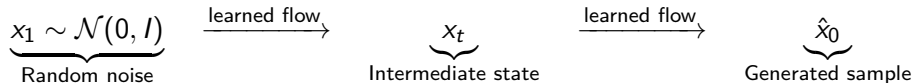
Black Forest Labs and MIT, ICML 2026

Presenter: Pengxi Liu

September 25, 2026

Flow matching: from noise to data

Objective: Learn a velocity field that gradually transforms random noise into samples from the data distribution.



Generation follows the trajectory from $t = 1$ to $t = 0$.

- **What the model predicts:** a velocity $f_\theta(x_t, t)$ describing how the current sample changes with time.
- **Training:** use data–noise pairs to teach the model the velocity at different points along a path.
- **Generation:** start from pure noise and numerically integrate the learned velocity field toward $t = 0$.

Flow matching: training input

We start with a clean sample x_0 and independent Gaussian noise ϵ . For images, x_0 is usually a sequence of latent tokens from an autoencoder.

$$x_t = (1 - t)x_0 + t\epsilon, \quad t \in [0, 1].$$

Time	What the model receives
$t = 0$	Clean data: $x_t = x_0$
$t = 0.5$	A mixture: $x_t = 0.5x_0 + 0.5\epsilon$
$t = 1$	Pure noise: $x_t = \epsilon$

Task: Given x_t and t , predict a velocity that can move the sample from noise toward data.

Flow matching: what the loss compares

The sampled interpolation has a known velocity:

$$u = \frac{dx_t}{dt} = \epsilon - x_0.$$

Quantity	Meaning
$\hat{u} = f_{\theta}(x_t, t, c)$	Velocity predicted from the noisy input
$u = \epsilon - x_0$	Training target computed from the sampled pair
c	Optional condition, such as a class label or text

$$\mathcal{L}_{\text{gen}} = \mathbb{E} \left[\left\| \underbrace{\hat{u}}_{\text{prediction}} - \underbrace{u}_{\text{target}} \right\|^2 \right].$$

The loss penalizes velocity prediction errors, averaged over samples, noise, and trains every layer of the network through backpropagation.

Where intermediate features enter the flow model

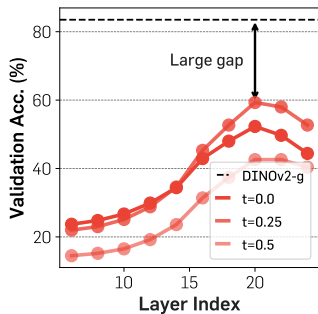
The flow network turns noisy tokens into velocity predictions:

$$x_t \xrightarrow{\text{early blocks}} H_{\theta}^{(\ell)} \xrightarrow{\text{remaining blocks}} \hat{u}.$$

Object	Role
$x_t \in \mathbb{R}^{N \times C}$	Noisy input tokens
$H_{\theta}^{(\ell)} \in \mathbb{R}^{N \times d}$	Information the network extracts at layer ℓ
$\hat{u} \in \mathbb{R}^{N \times C}$	Predicted change per unit time for every token

- The intermediate features learn semantic meanings indirectly, through the velocity matching.

Can feature supervision help the flow model learn faster?



(a) Semantic gap: Linear probing

- A simple linear classifier tests how easily object categories can be read from the hidden features.
- A flow-based generator (SiT), trained for 7M steps, still lags behind DINOv2, a pretrained visual encoder.
- **Hypothesis:** Could an extra feature target help the flow model learn a better velocity field with less training?

Reference: Yu, Sihyun, et al. "Representation alignment for generation: Training diffusion transformers is easier than you think." (2024)

REPA: adding a feature target to flow matching

REPA = REPresentation Alignment

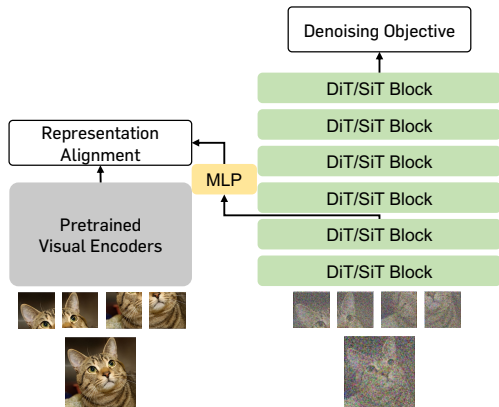
- A pretrained visual encoder, such as DINOv2, extracts target features from the clean image.
- The flow model sees the noisy input. A projection of its intermediate features learns to match those target features.

Training objective	Compared quantities	What it teaches
Flow loss	Predicted vs. target velocity	How the sample should move
Feature loss	Student vs. teacher features	An intermediate representation for that prediction

The model keeps learning the velocity field. REPA adds supervision inside the network.

Reference: Yu, Sihyun, et al. "Representation alignment for generation: Training diffusion transformers is easier than you think." (2024)

REPA: two training branches



Denoising branch

$$x_t \mapsto (H_{\theta}^{(\ell)}, \hat{u}), \quad \hat{u} \approx \epsilon - x_0.$$

Alignment branch

$$h_{\psi}(H_{\theta}^{(\ell)}) \approx \text{sg}[r_{\phi}(y)].$$

- h_{ψ} is a trainable projection MLP.
- The target uses the clean image paired with this noisy latent.
- Both losses update one generative model.

Reference: Yu, Sihyun, et al. "Representation alignment for generation: Training diffusion transformers is easier than you think." (2024)

REPA: a patchwise alignment objective

For spatial token i , compare two feature vectors:

$$\underbrace{z_i = h_\psi(H_\theta^{(\ell)}(x_t, t, c)_i)}_{\text{projected student feature}}, \quad \underbrace{T_i = \text{sg}[r_\phi(y)_i]}_{\text{frozen teacher feature from clean image } y}.$$

$$\mathcal{L}_{\text{REPA}} = -\mathbb{E} \frac{1}{N} \sum_{i=1}^N \underbrace{\frac{z_i^\top T_i}{\|z_i\|_2 \|T_i\|_2}}_{\text{cosine similarity}}.$$

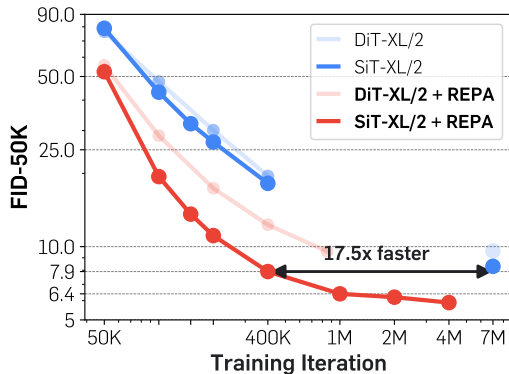
Minimizing the negative cosine similarity makes the feature directions more alike.

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{gen}}}_{\text{velocity prediction}} + \lambda \underbrace{\mathcal{L}_{\text{REPA}}}_{\text{feature alignment}}.$$

h_ψ matches feature dimensions. The teacher is fixed. λ controls the extra supervision.

Reference: Yu, Sihyun, et al. "Representation alignment for generation: Training diffusion transformers is easier than you think." (2024)

REPA: evidence of acceleration



Reference: Yu, Sihyun, et al. "Representation alignment for generation: Training diffusion transformers is easier than you think." (2024)

REPA: the limits of an external teacher

- **Task mismatch.** Features useful for visual recognition may leave out information needed for detailed synthesis.
- **Teacher dependence.** A suitable encoder must be chosen for each data domain or modality.
- **Uncertain gains.** In Self-Flow's study, stronger DINO variants do not produce better generations with REPA.

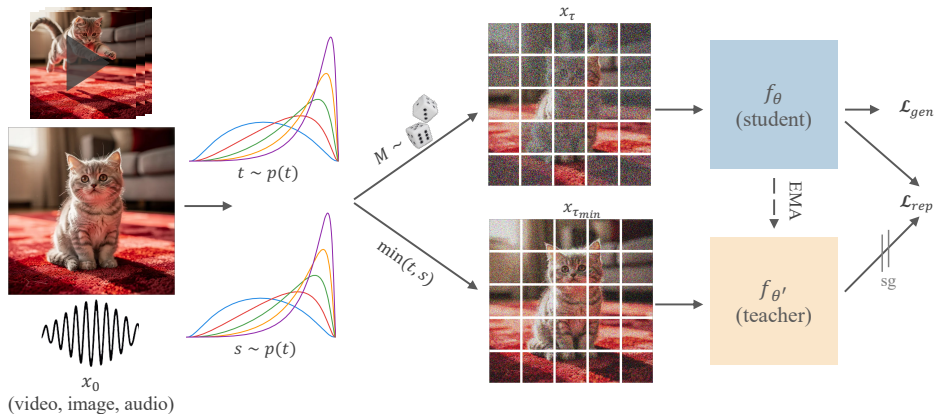
Question: How can we learn useful representation targets within generative training, without a separate pretrained feature teacher?

Self-Flow: learning targets within flow training

- **Internal teacher.** An EMA copy of the generator produces target features from a cleaner view of the same sample.
- **A useful prediction task.** The student sees a mix of cleaner and noisier tokens, and learns to predict the teacher's features while denoising.
- Cleaner regions provide context for interpreting noisier regions, encouraging the student to learn relations across tokens.

Why this addresses the problem: The target evolves with generative training. The same approach applies across modalities without choosing a separate feature encoder for each one.

Method: framework



Student: mixed-noise input

Teacher: all tokens use the lower noise level

Method: dual-timestep scheduling

1 Sample two noise levels.

Independently draw $t, s \sim p(t)$. These two values are shared across all tokens in this sample.

2 Randomly split the tokens into two groups.

Select each token independently with probability R_M . Let M be the set of selected tokens, chosen independently of t, s . Assign

$$\tau_i = \begin{cases} s, & i \in M \quad (\text{selected token}), \\ t, & i \notin M \quad (\text{unselected token}). \end{cases}$$

3 Add noise using each token's assigned level.

Sample one noise tensor $x_1 \sim \mathcal{N}(0, I)$ and construct

$$x_{\tau}^i = (1 - \tau_i)x_0^i + \tau_i x_1^i.$$

Method: dual-timestep scheduling

Intuition: use cleaner regions to understand noisier regions.

- Standard flow matching uses the same noise level for every token in a sample.
- With two noise levels, some regions are clearer than others. This encourages the model to use information across tokens.

Each token still follows the original noise-level distribution.

Draw t, s independently from the same distribution p . Each token uses t with probability $1 - R_M$ and s with probability R_M , independently of their values.

For any threshold $a \in [0, 1]$, let $F(a) = \Pr(t \leq a) = \Pr(s \leq a)$. Then

$$\begin{aligned}\Pr(\tau_i \leq a) &= (1 - R_M) \Pr(t \leq a) + R_M \Pr(s \leq a) \\ &= (1 - R_M)F(a) + R_M F(a) = F(a).\end{aligned}$$

The noise levels within a sample become heterogeneous, while each token's distribution over training remains unchanged.

Method: EMA teacher?

Problem: the prediction target is also changing. The student learns to predict features from another copy of itself. If we copy the latest student weights at every step, the target network changes immediately after every update.

Solution: Let the student learn through gradients, while the teacher follows more slowly. This reduces fluctuations in the feature targets caused by parameter updates.

Initialize $\theta' = \theta$. At each training iteration:

- 1 **Compute target features from the cleaner input:** $x_a = (1 - a)x_0 + ax_1, , \quad a = \min(t, s)$.
- 2 **Update the student using the training loss:**

$$\theta \leftarrow \text{OptimizerStep}(\theta, \nabla_{\theta} \mathcal{L}).$$

- 3 **Slowly update the teacher weights:**

$$\theta' \leftarrow \underbrace{0.9999 \theta'}_{\text{previous teacher}} + \underbrace{0.0001 \theta}_{\text{updated student}} .$$

EMA averages **weights**, not features. The next iteration recomputes features using the updated teacher.

Method: objective

Self-Flow obtains the feature target from its EMA flow model:

$$\underbrace{z_i = h_\psi(H_\theta^{(\ell)}(x_\tau, \tau)_i)}_{\text{student: mixed-noise input}}, \quad \underbrace{T_i = \text{sg}[H_{\theta'}^{(k)}(x_a, a)_i]}_{\text{teacher: cleaner input}}.$$

$$\mathcal{L}_{\text{rep}} = -\mathbb{E} \frac{1}{N} \sum_{i=1}^N \frac{z_i^\top T_i}{\|z_i\|_2 \|T_i\|_2}.$$

The student also predicts velocity from its mixed-noise input:

$$\mathcal{L}_{\text{gen}} = \mathbb{E} \|f_\theta(x_\tau, \tau) - (\epsilon - x_0)\|^2.$$

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{gen}}}_{\text{velocity prediction}} + \gamma \underbrace{\mathcal{L}_{\text{rep}}}_{\text{feature prediction}}, \quad \gamma = 0.8.$$

Method: alignment across network depth

For a network with depth D , the paper chooses

$$\ell \approx 0.3D, \quad k \approx 0.7D, \quad \ell < k.$$

- A shallower student feature predicts a deeper teacher feature.
- The feature loss reaches the student prefix and projection head. Later blocks train through the generation objective.
- For SiT-XL, the layer ablation uses $(\ell, k) = (8, 20)$ as its default.

Method: one training iteration

- 1 Encode a training sample to x_0 and sample x_1, t, s, M .
- 2 Build τ , student input x_τ , and teacher input $x_{\min(t,s)}$.
- 3 Run the student with per-token timestep conditioning. Retain its velocity prediction and layer- ℓ features.
- 4 Run the EMA teacher without gradients. Retain layer- k features.
- 5 Compute $\mathcal{L}_{\text{gen}} + 0.8\mathcal{L}_{\text{rep}}$ and update student weights.
- 6 Update the EMA weights.

Cost: an additional teacher forward pass during training.

Method: inference

- 1 **Start from random noise:** $x_1 \sim \mathcal{N}(0, I)$.
- 2 **Use the EMA model to predict the velocity.**

All tokens use the same timestep t :

$$\hat{u} = f_{\theta'}(x_t, t, c).$$

- 3 **Take a small step toward data.**

$$x_{t-\Delta t} \approx x_t - \Delta t \hat{u}, \quad \Delta t > 0.$$

Repeat steps 2–3 from $t = 1$ to $t = 0$.

- 4 **Decode the final latent** into an image, video, or audio sample.

Training versus inference: Training uses two noise levels; inference uses one. Dual-timestep scheduling limits this mismatch by keeping tokens in two shared-time groups and preserving each token's noise-level distribution. It does not eliminate the gap.

Comparison: REPA and Self-Flow

	REPA	Self-Flow
Student corruption	Homogeneous time t	Two tokenwise times t, s
Target network	Frozen visual encoder	EMA generative model
Target input	Clean image y	Cleaner latent $x_{\min(t,s)}$
Matched object	Projected hidden patch features	Projected hidden token features
Representation source	External pretraining	The model's own training
At inference	Generative network + decoder	Generative network (EMA) + decoder

Both retain the generation objective and add a training-time feature loss. Self-Flow changes both the target source and the corruption task.

Experiments: setup

Task	Backbone	Latent encoder	Training data
Class-to-image	SiT-XL	SD-VAE / RAE	ImageNet-1K
Text-to-image	FLUX.2-based	SD-VAE	20M image-text pairs
Text-to-video	FLUX.2-based	Wan2.2 AE	6M videos
Text-to-audio	FLUX.2-based	Songbloom AE	1M audio clips

Experiments: ImageNet generation

Method	Steps	FID ↓	sFID ↓	IS ↑
SiT-XL/2	7M	8.30	6.30	130.57
SRA	4M	7.27	5.87	143.06
REPA	4M	5.89	5.73	157.66
Self-Flow	4M	5.70	4.97	151.40

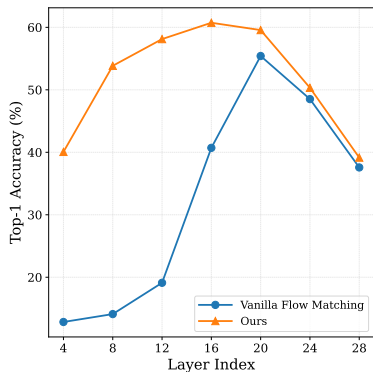
- Self-Flow improves FID over REPA at 4M steps, with no external alignment teacher.
- The improvement is not uniform across every metric: REPA has higher IS.
- On ImageNet, Self-Flow overtakes REPA only after approximately 3M steps (App. G.1).

Experiments: text-to-image generation

Method (1M steps)	FID ↓	FD-DINO ↓	CLIP ↑
Vanilla flow	4.08	204.49	30.66
SRA	3.70	176.79	30.78
REPA	3.92	173.35	30.67
SigLIP 2 alignment	3.97	196.75	30.68
Self-Flow	3.61	167.98	30.88

Self-Flow also improves the distance measured in DINOv2 feature space, although REPA explicitly aligns to DINOv2.

Experiments: linear probing



(b) Linear probing on ImageNet

Freeze the trained flow model and fit a linear classifier on its hidden features.

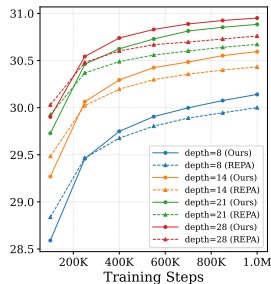
$$\text{class scores} = Wz + b.$$

- This figure measures ImageNet top-1 accuracy at different layers after 2M training steps.
- Early and middle layers improve substantially.

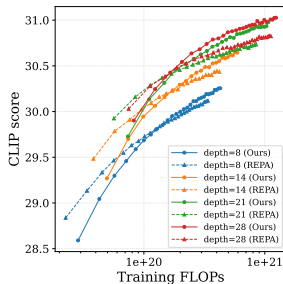
Experiments: video generation

Method (600K steps)	FVD ↓	Frame FID ↓
Vanilla flow	50.95	9.28
SRA	49.75	9.02
DINOv2 alignment	49.59	9.39
Depth Anything 3 alignment	51.52	9.85
V-JEPA2 alignment	53.55	9.91
Self-Flow	47.81	8.92

Experiments: model size and training compute



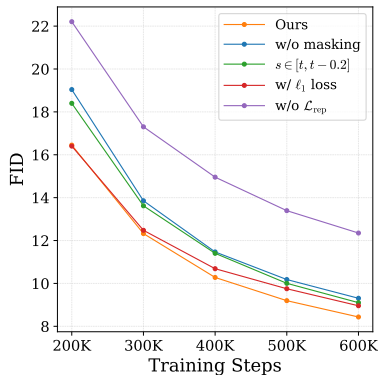
(a) Steps



(b) FLOPs

- Model sizes: 290M, 420M, 625M, and 1B parameters.
- Self-Flow gains more from increasing model size in this study.
- The 625M Self-Flow model exceeds 1B REPA on the plotted CLIP score.
- The FLOP plot accounts for training compute, beyond iteration counts.

Ablation: which components matter?



(a) Ablation study

- **Remove $\mathcal{L}_{rep}$:** largest FID degradation, over 4 points.
- **Remove dual-timestep masking:** substantial degradation even with feature alignment.
- **Keep the second time only slightly cleaner:** weaker results.
- **Replace cosine with ℓ_1 :** late instability associated with feature-norm growth.

Is it worth reading? Yes!

- Connect representation learning with flow matching: can better intermediate features help generative training?
- Provide a simple alternative to external feature alignment methods like REPA.

Is it worth implementing? Maybe.

- They don't provide training code yet!
- For my research: test whether it can be adopted in language settings.